

CENG 495

Cloud Computing

2025-2026 Spring

Take Home Exam 3- Distributed Data Processing

(Ver. 1.1)

May 25 2026, Monday 23:59

1 Introduction

In this assignment, you will design several distributed algorithms capable of processing massive datasets. The tasks are designed to teach you how to handle network bottlenecks, distribute workloads, force data partitioning, and build automated analytical pipelines.

2 Dataset

For the assignment, your code will operate on the `ratings.csv` file in the **MovieLens Small Dataset**¹.

Data format of the dataset is as follows:

`userId,movieId,rating,timestamp`

(e.g., 1,31,2.5,1260759144)

3 Language and the Framework

For the solution you will use **Python**² and its **mrjob**³ framework.

You can find the template on last page of the PDF for your tasks. Note that **JOBCONF must be identical** to the template for every task.

4 Tasks

You can find the 5 tasks you need to solve under this section.

Global architectural constraints for **all tasks** are:

- **Handle the Header:** Your dataset contains a header row (`userId, movieId, rating, timestamp`). Your code must handle or skip this header to avoid data type errors.
- **Force Multiple Map Tasks:** The provided dataset is small and would normally be processed by a single Map Task. To simulate a true distributed environment, you must

¹<https://grouplens.org/datasets/movielens/>

²<https://www.python.org/>

³<https://mrjob.readthedocs.io/en/latest/>

configure your jobs to split the input file into 1MiB chunks (1048576 bytes). This will force the framework to launch multiple independent Map tasks.

- **Maximize Parallelism:** Wherever mathematically and logically sound, your aggregation phases must be distributed across multiple (e.g., 4) reducer instances.
- **Minimize Traffic:** Network I/O is expensive. Wherever mathematically and logically sound, you must implement a mechanism that pre-aggregates the data locally on the mapping nodes before transmitting it to the reducers.

Task 1

Objective:

Determine the total number of each rating value across the entire platform.

Expected Output Format:

```
rating  total_number
```

Example Output:

```
0.5      1370
1.0      2811
4.0      26818
...
```

Task 2

Objective:

Calculate the average rating for every movie.

Expected Output Format:

```
movieId  average_rating
```

Example Output:

```
8782  2.75
8783  3.19
87834 5.0
...
```

Task 3

Objective:

Identify the platform's "Hidden Gems". A movie is a hidden gem if:

1. It has received between 30 and 100 votes (inclusive).
2. Its average rating is 4.0 or higher.

Output all the hidden gems ordered globally by the average rating descending; and if tied, by total votes descending.

Expected Output Format:

```
movieId  [average_rating,  total_votes]
```

Example Output:

```
1204    [4.3, 45]
1276    [4.27, 57]
750     [4.27, 97]
...
```

Task 4

Objective:

Analyze the number of ratings per month (convert the UNIX timestamp into YYYY-MM).

Expected Output Format:

```
"YYYY-MM"  total_number_of_ratings
```

Example Output:

```
"2018-09"      604
"2017-01"      193
"2017-02"      420
...
```

Task 5

Objective:

Identify “Harsh Critics”. A user is a harsh critic if:

1. They have rated at least 50 movies.
2. Their personal average given rating is strictly less than 2.5.

Output all of these users, sorted globally by their average rating in ascending order (most strict critics first).

Expected Output Format:

```
userId  [average_given_rating,  total_ratings_given]
```

Example Output:

```
139     [2.14, 194]
153     [2.22, 179]
567     [2.25, 385]
...
```

5 Testing and Debugging Your Code: Local Simulation

You **do not** need to install a full Hadoop cluster to develop, test, and debug your MapReduce jobs. The **mrjob** framework is designed to simulate a distributed environment directly on your personal computer using your local CPU and RAM.

5.1 The Execution Command

To execute a task run the following command in your terminal:

```
python task#.py ratings.csv > final_output.txt
```

where # is the task number.

5.2 What happens behind the scenes?

When you run the command, **mrjob** automatically operates in Local Mode. It will read your **JOBCONF** settings (like the 1MiB split size and the 4 reducer instances) and use multiple local processes to simulate the Map and Reduce tasks. It is the standard industry practice to verify mathematical logic and pipeline flow before deploying code to a massive, real-world cloud clusters.

6 Rules and Submission

- This is an exam. Plagiarism will not be tolerated.
- Upload a **zip** file containing all your task solution files to ODTUClass assignment page before the deadline.
 - The penalty for late submission is $(\text{Number of Late Days})^2 \times 5$.
 - You will get 0 automatically after 3 late days.
- Your task solutions must be **Python files** and should be named according to the task number: **task1.py** for the first task, **task2.py** for the second task etc.

General Purpose mrjob Template

```
from mrjob.job import MRJob
from mrjob.step import MRStep
import csv

class AdvancedMRPipeline(MRJob):

    # GLOBAL CONFIGURATION FOR THIS JOB
    JOBCONF = {
        # Force the framework to split input
        # files into 1MiB chunks (1048576 bytes)
        'mapreduce.input.fileinputformat.split.maxsize': '1048576',
        # Set default parallel reducers for aggregation phases
        'mapreduce.job.reduces': '4'
    }

    def steps(self):
        return [
            # PHASE 1: Parallel Aggregation
            # (Uses the global 4 reducers)
            MRStep(
                mapper=self.mapper_phase1,
                combiner=self.combiner_phase1,
                reducer=self.reducer_phase1
            ),
            # PHASE 2: Global Sorting
            # (Overrides global config to use exactly 1 reducer)
            MRStep(
                jobconf={'mapreduce.job.reduces': '1'},
                reducer=self.reducer_phase2_sort
            )
            # Note that mrjob does not support overriding
            # the default partitioner
        ]

    # ... Define your mapper, combiner, and reducer functions below ...

if __name__ == '__main__':
    AdvancedMRPipeline.run()
```